PROGRAMMING IN JAVA

ASSIGNMENT 2 – CO2

INTRODUCTION

This assignment studies the practical use of Java strings, collection classes, hashing, maps, generics, comparators and priority queues. Each problem is based on a real application such as railway ticket generation, music playlists, event attendance, pharmacy inventory, sports ranking and hospital triage.

The work follows an engineering process: predict the behaviour, design test cases, implement the solution, measure its performance, analyse the results and evaluate the design. The programs supplied with this report are separate Java source files so that each task can be compiled and executed independently.

OBJECTIVES

- Compare String, StringBuilder and StringBuffer for repeated text construction.
- Evaluate ArrayList, LinkedList and ArrayDeque for different list operations.
- Apply the equals() and hashCode() contract correctly in hash-based collections.
- Choose suitable Map implementations for lookup and ordered reporting.
- Develop reusable generic ranking logic with Comparator.
- Implement priority-based hospital triage while preserving arrival order.
- Diagnose a performance limitation and validate a redesigned solution.

EXECUTION ENVIRONMENT AND METHOD

Recommended environment: Java 8 or later (Java 17 is suitable), a JDK, and a terminal or IDE such as IntelliJ IDEA, Eclipse or VS Code. Save each source file using the exact public class filename. Compile using javac FileName.java and run using java ClassName.

For credible benchmarking, run each test several times, perform warm-up iterations, record System.nanoTime(), calculate an average, and repeat at three or more input sizes. The values printed by

the programs depend on the computer, JVM, operating system and background load. Therefore, enter your own measured values in the result tables after execution.

Item	Details
Java version	Record using: java -version
Processor / RAM	Fill in your laptop specifications
Operating system	Record your OS version
Measurement	System.nanoTime(); repeat and average
Evidence	Save console screenshots and raw timing values

Q1 – BUILDING TEXT UNDER LOAD: STRING, STRINGBUILDER OR STRINGBUFFER?

Answer and analysis

String is immutable. Each concatenation creates a new String value and copies the previous contents into it. Repeated concatenation inside a loop can therefore copy an increasing amount of text. StringBuilder is mutable and appends to an internal buffer, reducing intermediate String creation. StringBuffer offers similar append operations but its methods are synchronized, making individual operations thread-safe with additional synchronization overhead.

Approach	Expected behaviour	Suitable use
String + in loop	Repeated objects and repeated copying; costly as fields grow	Small, fixed text expressions
StringBuilder	Efficient append operations; not synchronized	Per-ticket text built by one thread
StringBuffer	Synchronized append operations; usually slower than Builder	Shared mutable text only when synchronized methods are sufficient

Prediction: for one short ticket, the difference may be small. For thousands of tickets containing many fields, StringBuilder should usually be faster and create fewer temporary strings. StringBuffer may be slower than StringBuilder because of synchronization.

Workloads and measurements

Workload A: one ticket containing 5 fields. Workload B: 10,000 tickets containing 50 fields each. Workload C: 1,000 tickets containing 200 fields each. Run the benchmark repeatedly for each workload. The included Q1 program measures the three construction methods for 50 fields; change FIELDS and repetitions to test the other workloads.

Workload	Fields/ticket	Tickets	String + avg ns	Builder avg ns	Buffer avg ns
A	5	1	Fill after run	Fill after run	Fill after run
B	50	10,000	Fill after run	Fill after run	Fill after run
C	200	1,000	Fill after run	Fill after run	Fill after run

Design decision and complexity

Use StringBuilder to construct each ticket because each ticket is built locally by one request thread. For a shared booking log, use a dedicated logging framework or a synchronized/concurrency-safe logging design rather than allowing many threads to mutate one StringBuilder. If a shared in-memory buffer is specifically required, StringBuffer provides synchronized methods, although compound operations may still need external synchronization.

For n fields and a total output length L, repeated concatenation can require $O(L^2)$ character copying in the worst repeated-prefix model. StringBuilder append is amortized O(L) overall, with O(L) output/buffer space. StringJoiner can also assemble delimited fields and String.join can join a collection of strings; String.format is convenient for formatted output but is not generally the fastest choice for repeated construction.

Intermediate strings in repeated concatenation increase temporary allocation and garbage-collection work. StringBuilder mainly uses one growable character buffer plus the final String returned by toString().

PROGRAM:

```
public class Q1_StringBenchmark {
   static int FIELDS = 50;
   static volatile int result;
      static String usingString() {
      String text = "";
      for (int i = 0; i < FIELDS; i++) {
         text = text + "Field" + i + ":Value;";
      }
      return text;
   }
   // Using StringBuilder
   static String usingBuilder() {
      StringBuilder text = new StringBuilder();
      for (int i = 0; i < FIELDS; i++) {
         text.append("Field").append(i).append(":Value;");
      }
      return text.toString();
   }
   // Using StringBuffer
   static String usingBuffer() {
      StringBuffer text = new StringBuffer();
      for (int i = 0; i < FIELDS; i++) {
         text.append("Field").append(i).append(":Value;");
      }
      return text.toString();
   }
   static long measure(Runnable task) {
      long start = System.nanoTime();
      for (int i = 0; i < 100; i++) {
         task.run();
      }
      return (System.nanoTime() - start) / 100;
   }
   public static void main(String[] args) {
      for (int i = 0; i < 100; i++) {
         usingString();
         usingBuilder();
         usingBuffer();
      }
      long t1 = measure(() ->
         result = usingString().length());
```

```
        long t2 = measure(() ->
            result = usingBuilder().length());
        long t3 = measure(() ->
            result = usingBuffer().length());
        System.out.println("String Time: " + t1 + " ns");
        System.out.println("StringBuilder Time: " + t2 + " ns");
        System.out.println("StringBuffer Time: " + t3 + " ns");
        System.out.println("Ticket Generated Successfully");
    }
}
```



Q2 – WHEN IS ARRAYLIST AN APPROPRIATE ENGINEERING CHOICE?

Answer and analysis

ArrayList stores elements in a resizable array. It offers constant-time indexed access and amortized constant-time append. Inserting or removing at the front or middle requires shifting later elements, so the operation is O(n). LinkedList stores separate nodes; insertion/removal at a known iterator position is O(1), but locating an index requires traversal and is O(n). ArrayDeque is a resizable circular-array deque and is well suited to adding and removing at both ends.

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	O(1) amortized	O(1)	O(1) amortized at either end

Insert at front	O(n)	O(1)	O(1)
Insert in middle	O(n) shifting	O(n) locate; O(1) link	Not designed for indexed middle insertion
Remove at front	O(n)	O(1)	O(1)
Get k-th element	O(1)	O(n)	No indexed access; O(n) traversal
Full iteration	O(n), cache-friendly	O(n)	O(n)

ArrayList is a reasonable choice when the playlist frequently jumps to a selected song, appends songs and iterates through the playlist, while front/middle modifications are relatively uncommon. Stop recommending it when frequent insertions and removals near the front dominate. For an Up Next queue with frequent operations at both ends, ArrayDeque is generally more appropriate.

A plain array with manual shifting has O(1) indexed access and O(n) insertion/removal away from the end. It also requires capacity management and cannot grow automatically.

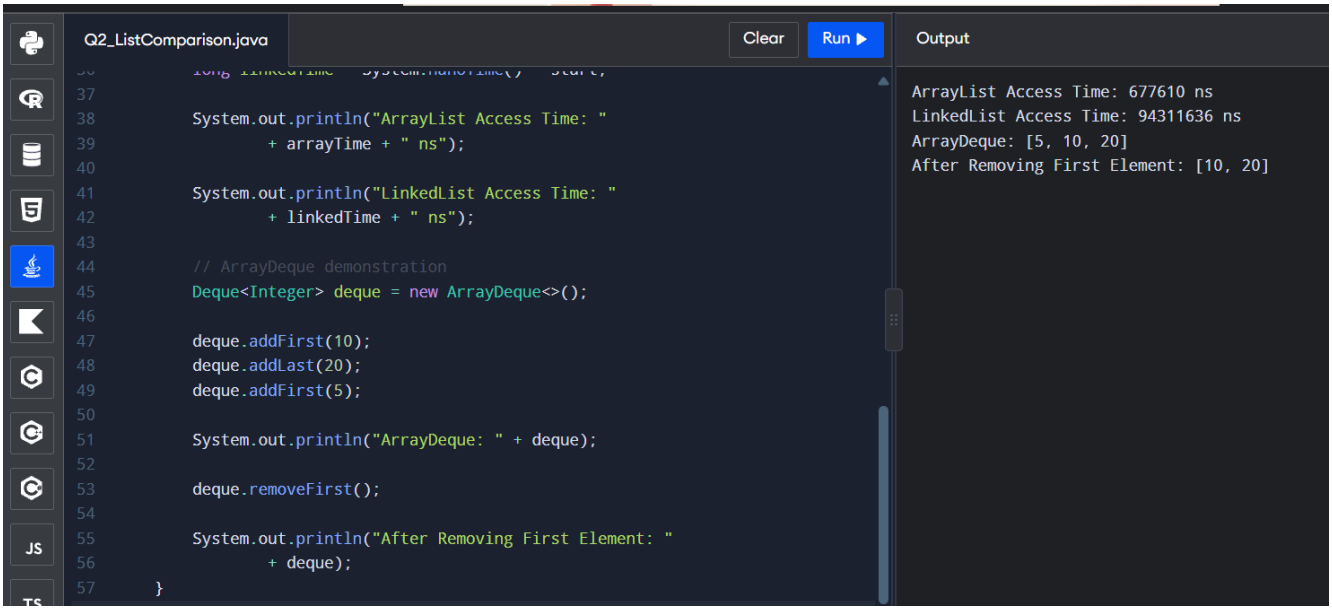
Experimental design

Use playlist sizes 1,000, 10,000 and 50,000. Compare append, front insertion, middle insertion/removal, middle-index access and full iteration. The supplied Q2 program measures middle access, iteration and front insertion, and demonstrates ArrayDeque at both ends. Extend it with repeated trials for middle insertion/removal and append to complete the full experiment.

Size	**Operation**	**ArrayList ns**	**LinkedList ns**	**Notes**
1,000	Get middle	Fill after run	Fill after run	Repeat 10,000 times
10,000	Iteration	Fill after run	Fill after run	One full traversal
50,000	Insert front	Fill after run	Fill after run	Use identical conditions

PROGRAM

```
import java.util.*;
public class Q2_ListComparison {
   static volatile int result;
   public static void main(String[] args) {
      List<Integer> arrayList = new ArrayList<>();
      List<Integer> linkedList = new LinkedList<>();
      int size = 10000;
      // Adding elements
      for (int i = 0; i < size; i++) {
         arrayList.add(i);
         linkedList.add(i);
      }
      // ArrayList access
      long start = System.nanoTime();
      for (int i = 0; i < 10000; i++) {
         result = arrayList.get(5000);
      }
      long arrayTime = System.nanoTime() - start;
      // LinkedList access
      start = System.nanoTime();
      for (int i = 0; i < 10000; i++) {
         result = linkedList.get(5000);
      }
      long linkedTime = System.nanoTime() - start;
      System.out.println("ArrayList Access Time: "
            + arrayTime + " ns");
      System.out.println("LinkedList Access Time: "
            + linkedTime + " ns");
     // ArrayDeque demonstration
      Deque<Integer> deque = new ArrayDeque<>();
      deque.addFirst(10);
      deque.addLast(20);
      deque.addFirst(5);
      System.out.println("ArrayDeque: " + deque);
      deque.removeFirst();
      System.out.println("After Removing First Element: "
            + deque);
   }
}
```



Q3 – ENGINEER THE HASHCODE–EQUALS CONTRACT

Answer and analysis

HashSet uses hashCode() to identify a bucket and then uses equals() to distinguish objects in that bucket. If two objects are equal according to equals(), they must return the same hash code. The reverse is not required: unequal objects may have the same hash code.

The original Student class overrides equals() but not hashCode(). Two Student objects with the same registration number may therefore have different identity-based hash codes and be placed in different buckets. HashSet may fail to recognise them as duplicates.

Test	**Data / action**	**Expected result after correction**
1 – duplicate scan	Add Arun ID 001 twice and Divya ID 002 once	Set size = 2
2 – repeated scan	Add the same ID from multiple gates	Only one entry per registration number

3 – immutability check	Change the registration number after insertion (unsafe design)	Avoid mutable equality keys; changing hash fields can make lookup/removal fail

The corrected class uses a final registration number and implements hashCode() using the same field used by equals(). The included code prints two unique students. It also demonstrates alternative de-duplication structures: TreeSet with a registration-number comparator and HashMap keyed by registration number.

A TreeSet with a Comparator on regNo de-duplicates in O(log n) per insertion and keeps sorted order. A HashMap<String, Student> keyed by regNo provides average O(1) insertion/lookup and stores one student per key. Both use O(n) space for n unique students.

Completed code output

Unique students: 2

TreeSet unique students: 2

HashMap unique students: 2

PROGRAM:

```
import java.util.*;
public class Q3_Attendance {
   // Student class
   static class Student {
      private final String regNo;
      private final String name;
      Student(String regNo, String name) {
         this.regNo = regNo;
         this.name = name;
      }
      public boolean equals(Object obj) {
         if (this == obj)
            return true;
         if (!(obj instanceof Student))
            return false;
         Student s = (Student) obj;

         return regNo.equals(s.regNo);
```

```
    }
    public int hashCode() {
        return Objects.hash(regNo);
    }
    public String toString() {
        return regNo + " " + name;
    }
  }
  public static void main(String[] args) {
    Set<Student> present = new HashSet<>();
    present.add(new Student("192311001", "Arun"));
    present.add(new Student("192311002", "Divya"));
    present.add(new Student("192311001", "Arun"));
    System.out.println("ATTENDANCE DETAILS");
    for (Student s : present) {
        System.out.println(s);
    }
    System.out.println("Unique Students: "
            + present.size());
  }
}
```

Q3_Attendance.java

Clear | Run

```
        }
    }

    public static void main(String[] args) {

        Set<Student> present = new HashSet<>();

        present.add(new Student("192311001", "Arun"));
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun"));

        System.out.println("ATTENDANCE DETAILS");

        for (Student s : present) {
            System.out.println(s);
        }

        System.out.println("Unique Students: "
                + present.size());
    }
}
```

Output

```
ATTENDANCE DETAILS
192311001 Arun
192311002 Divya
Unique Students: 2
```

Q4 – CHOOSING A MAP FOR A PHARMACY INVENTORY

Answer and analysis

A Map stores key-value pairs. The medicine code is the key and stock quantity is the value. HashMap is suitable for fast average lookup but does not guarantee iteration order. TreeMap sorts entries by key. LinkedHashMap preserves insertion order.

Map	**put/get average or typical cost**	**Iteration order**	**Best report**
HashMap	O(1) average; worst cases may be higher	No guaranteed order	Fast code lookup
TreeMap	O(log n)	Sorted by key	Medicine code order
LinkedHashMap	O(1) average	Insertion order	Order medicines were added

The pharmacy can maintain a HashMap for fast lookup and use TreeMap or LinkedHashMap for the required reports. If only one structure is desired, TreeMap naturally produces the code-sorted report but lookup is O(log n). LinkedHashMap naturally produces insertion order. HashMap requires sorting keys before a code-order report.

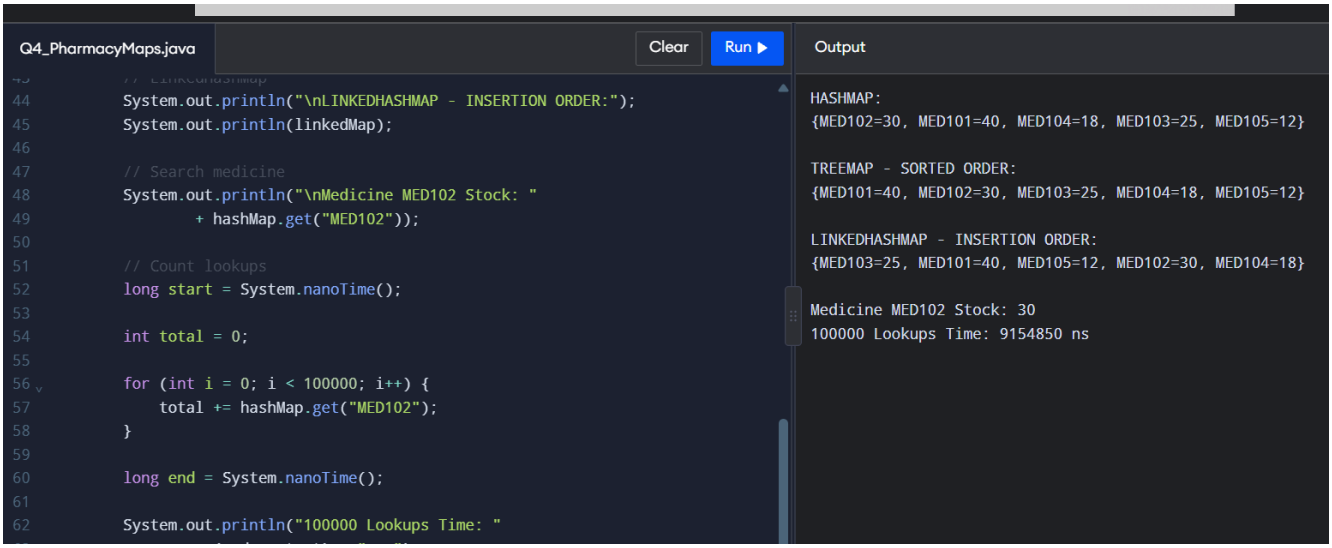
For three experiment sizes, use 100, 10,000 and 100,000 medicines. Measure 100,000 lookups and report generation separately. Define an acceptable single lookup target, for example less than 5 ms in the local test environment, and state that the target is an engineering requirement rather than a universal guarantee.

Hashtable is generally not preferred in new code because it is a legacy synchronized map. Modern code typically uses HashMap, ConcurrentHashMap or another collection selected for the concurrency requirement.

A sorted array plus Arrays.binarySearch() gives O(log n) lookup and O(n) insertion because elements may need shifting. Producing the code-order report is O(n) because the array is already sorted. It uses O(n) space and can be efficient for mostly static inventories.

PROGRAM:

```
import java.util.*;
public class Q4_PharmacyMaps {
   public static void main(String[] args) {
      Map<String, Integer> hashMap = new HashMap<>();
      Map<String, Integer> treeMap = new TreeMap<>();
      Map<String, Integer> linkedMap =
            new LinkedHashMap<>();
      // Medicine details
      String[][] medicines = {
         {"MED103", "25"},
         {"MED101", "40"},
         {"MED105", "12"},
         {"MED102", "30"},
         {"MED104", "18"}
      };
      // Insert medicine records
      for (String[] medicine : medicines) {
         String code = medicine[0];
         int quantity = Integer.parseInt(medicine[1]);
         hashMap.put(code, quantity);
         treeMap.put(code, quantity);
         linkedMap.put(code, quantity);
      }
      // HashMap
      System.out.println("HASHMAP:");
      System.out.println(hashMap);
      // TreeMap
      System.out.println("\nTREEMAP - SORTED ORDER:");
      System.out.println(treeMap);
      // LinkedHashMap
      System.out.println("\nLINKEDHASHMAP - INSERTION ORDER:");
      System.out.println(linkedMap);
      // Search medicine
      System.out.println("\nMedicine MED102 Stock: "
            + hashMap.get("MED102"));
      // Count lookups
      long start = System.nanoTime();
      int total = 0;
      for (int i = 0; i < 100000; i++) {
         total += hashMap.get("MED102");
      }
      long end = System.nanoTime();
      System.out.println("100000 Lookups Time: "
            + (end - start) + " ns");
   }
}
```



Q5 – DERIVE A GENERIC RANKING COMPONENT

Answer and analysis

A separate Comparator is preferable for a reusable leaderboard because teams, players, colleges and quiz teams may have different ranking rules. Comparable defines one natural ordering inside a class; Comparator allows multiple external ordering strategies without changing the model class.

The required order is points descending, goal difference descending, and team name ascending. Comparator.comparingInt(...).reversed() handles the descending points rule; thenComparing adds the next criterion; the final thenComparing by name provides alphabetical order.

Leaderboard<T> stores generic items and receives Comparator<? super T>. The ranking() method copies the stored list, sorts the copy and returns it, so the original insertion order is not changed. List<? extends T> is useful when accepting a list of T or a list of a subtype of T.

The comparator is transitive because it applies a lexicographic ordering: the first differing criterion decides the order, and all criteria use consistent integer or string comparisons. Bisons ranks above Panthers because both have 9 points and goal difference 4, so the final alphabetical comparison places 'Bisons' before 'Panthers'.

Sorting once costs O(n log n) time and O(n) extra space for the copied list (plus sorting implementation overhead). Three separate stable sorts also require O(n log n) per sort, or O(3n log n), and careful ordering of the stable sorts. A Comparator chain is clearer and easier to reuse.

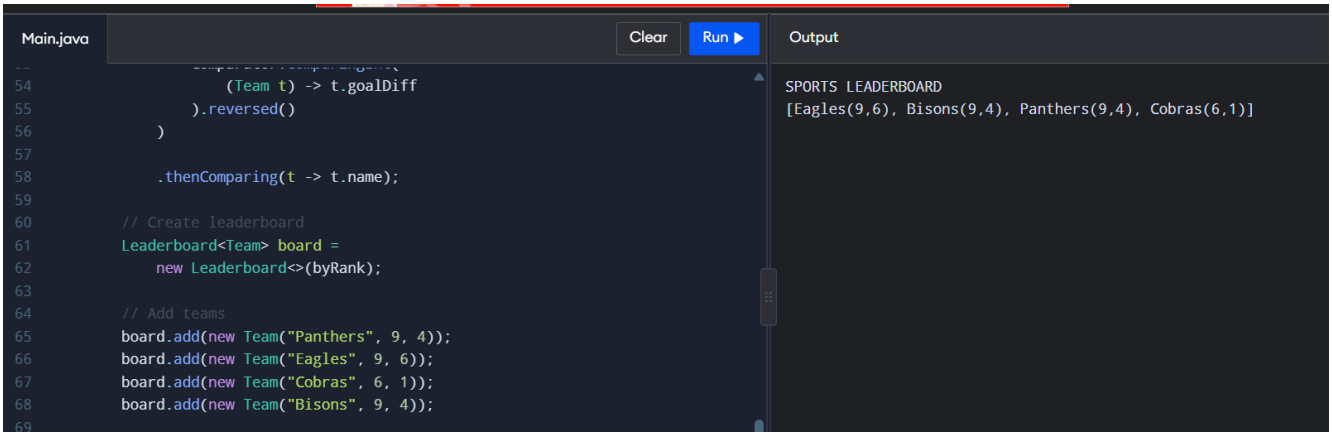
Expected output

[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]

PROGRAM:

```
import java.util.*;
class Team {
    final String name;
    final int points;
    final int goalDiff;
    Team(String name, int points, int goalDiff) {
        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }
    public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}
class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;
    Leaderboard(Comparator<? super T> order) {
        this.order = order;
    }
    void add(T item) {
        items.add(item);
    }
```

```
    List<T> ranking() {
        List<T> copy = new ArrayList<>(items);
        copy.sort(order);
        return copy;
    }
}
public class Q5_LeaderboardDemo {
    public static void main(String[] args) {
        // Ranking comparator
        Comparator<Team> byRank =
            Comparator.comparingInt(
                (Team t) -> t.points
            ).reversed()
            .thenComparing(
                Comparator.comparingInt(
                    (Team t) -> t.goalDiff
                ).reversed()
            )
            .thenComparing(t -> t.name);
        // Create leaderboard
        Leaderboard<Team> board =
                new Leaderboard<>(byRank);
        // Add teams
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));
        // Display ranking
        System.out.println("SPORTS LEADERBOARD");
        System.out.println(board.ranking());
    }
}
```



Q6 – PRIORITY, ORDER AND FAIRNESS IN A QUEUE

Answer and analysis

A normal FIFO queue serves the earliest arrival first, regardless of medical severity. It cannot guarantee that the most critical patient is served next. PriorityQueue orders patients by severity. Since severity 1 is most critical, the comparator sorts severity in ascending numeric order. A monotonically increasing arrival counter is used as the second comparison key so that patients with equal severity are served in arrival order.

Insertion and removal from PriorityQueue take O(log n) time; inspecting the head takes O(1). The queue stores n patients and therefore uses O(n) space. Printing PriorityQueue directly does not guarantee sorted treatment order because its iterator exposes the heap's internal arrangement, not a sorted traversal. Repeatedly poll a copy to display treatment order.

Starvation can occur when lower-priority patients continually arrive before a severity-5 patient. A fair policy can increase priority as waiting time passes (aging), or reserve service capacity for patients waiting beyond a threshold. Any aging policy must preserve urgent-care safety.

For 5,000 arrivals per day, a heap-based queue is technically scalable because each admission and removal is logarithmic. Deployment still requires concurrency control, persistence, monitoring, clinical validation and emergency override rules; daily volume alone is not sufficient to certify a hospital system.

Design	Admission	Serve next	Space
PriorityQueue	O(log n)	O(log n)	O(n)

Sorted LinkedList	O(n) search/insertion	O(1) at head	O(n) nodes
Five FIFO queues	O(1)	O(1) to check five heads	O(n)

Expected output

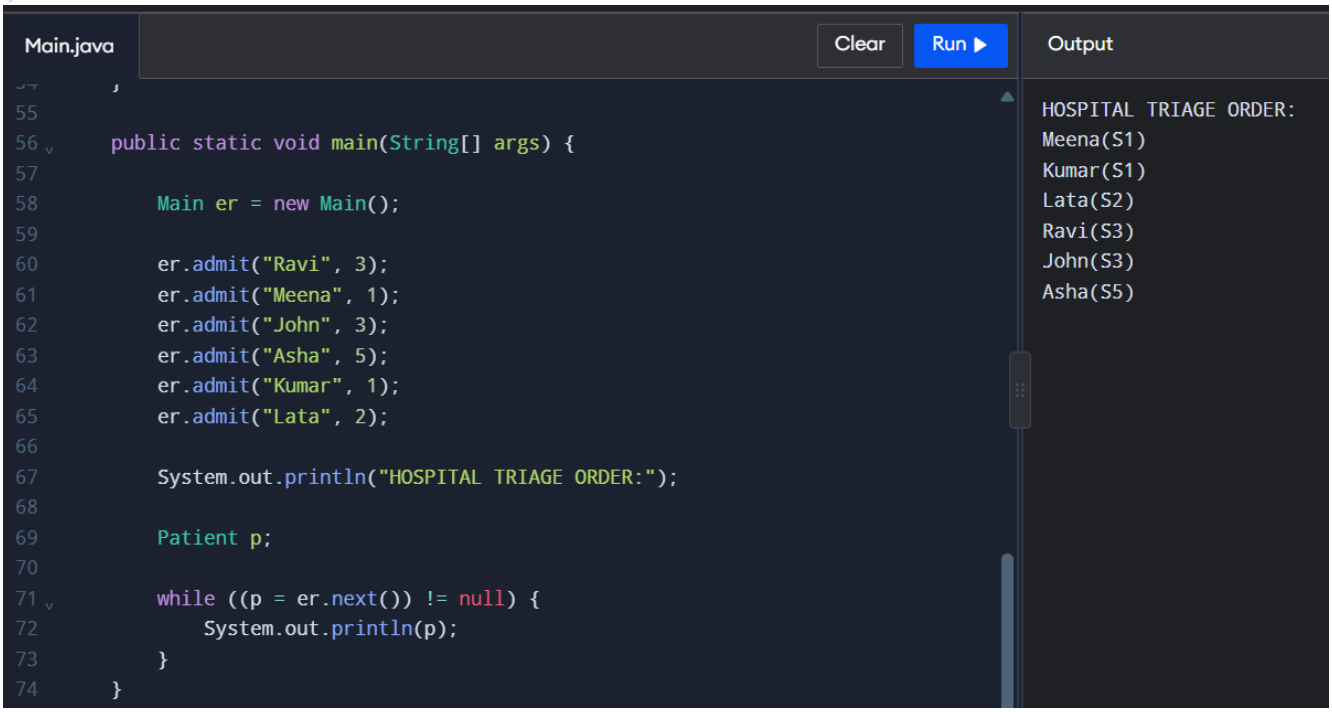
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)

The arrival counter preserves FIFO order within a severity level: Meena arrived before Kumar, and Ravi arrived before John.

PROGRAM:

```
import java.util.*;
public class Main {
   // Patient class
   static class Patient {
      final String name;
      final int severity;
      final long arrival;
      Patient(String name, int severity, long arrival) {
         this.name = name;
         this.severity = severity;
         this.arrival = arrival;
      }
      public String toString() {
         return name + "(S" + severity + ")";
      }
   }
   private long counter = 0;
   // PriorityQueue declaration
   private PriorityQueue<Patient> queue =
      new PriorityQueue<>(
         Comparator.comparingInt(
            (Patient p) -> p.severity
         ).thenComparingLong(
            p -> p.arrival
         )
      );
   // Admit patient
   void admit(String name, int severity) {
      if (severity < 1 || severity > 5) {
         throw new IllegalArgumentException(
            "Severity must be between 1 and 5"
```

```
            );
        }
        queue.add(
            new Patient(name, severity, counter++)
        );
    }
    // Remove next patient
    Patient next() {
        return queue.poll();
    }
    public static void main(String[] args) {
        Main er = new Main();
        er.admit("Ravi", 3);
        er.admit("Meena", 1);
        er.admit("John", 3);
        er.admit("Asha", 5);
        er.admit("Kumar", 1);
        er.admit("Lata", 2);
        System.out.println("HOSPITAL TRIAGE ORDER:");
        Patient p;
        while ((p = er.next()) != null) {
            System.out.println(p);
        }
    }
}
```



Q7 – DIAGNOSE, REDESIGN AND VALIDATE

Selected limitation: repeated String concatenation in Q1

A suitable redesign case is the railway ticket generator using repeated String concatenation. The bottleneck is repeated copying of the growing prefix and creation of intermediate immutable String objects. As the number of fields grows, the amount of copied text increases rapidly.

A. Diagnose the bottleneck

The operation that becomes expensive is appending each new field. Each concatenation creates a new String containing the old text plus the new field. The earlier content is copied repeatedly. This increases CPU work, temporary allocation and garbage collection.

B. Design objective

Replace repeated immutable concatenation with mutable buffered appends. The measurable objective is to reduce average construction time for large tickets and reduce temporary intermediate String creation, while preserving exactly the same ticket output.

C. Revised design

The redesigned method creates one StringBuilder, appends each field to it and calls toString() once at the end. The builder grows its internal buffer when needed. The trade-off is a mutable temporary buffer and a final String allocation.

Pseudocode: create StringBuilder; for each field append field; return builder.toString().

D. Prediction

For a small number of fields, the difference may be minor. For hundreds or thousands of fields, StringBuilder should require less copying and run faster than repeated concatenation.

E. Implement and validate

Run Q7_Redesign.java using identical field counts and repetitions for both methods. It tests 10, 100, 500 and 1,000 fields. Repeat the complete program several times, record the output, and compare the average times. Do not compare one isolated run only.

Fields	Original time (ns)	Redesigned time (ns)	Improvement / observation
10	Fill after run	Fill after run	Fill after run
100	Fill after run	Fill after run	Fill after run
500	Fill after run	Fill after run	Fill after run
1,000	Fill after run	Fill after run	Fill after run

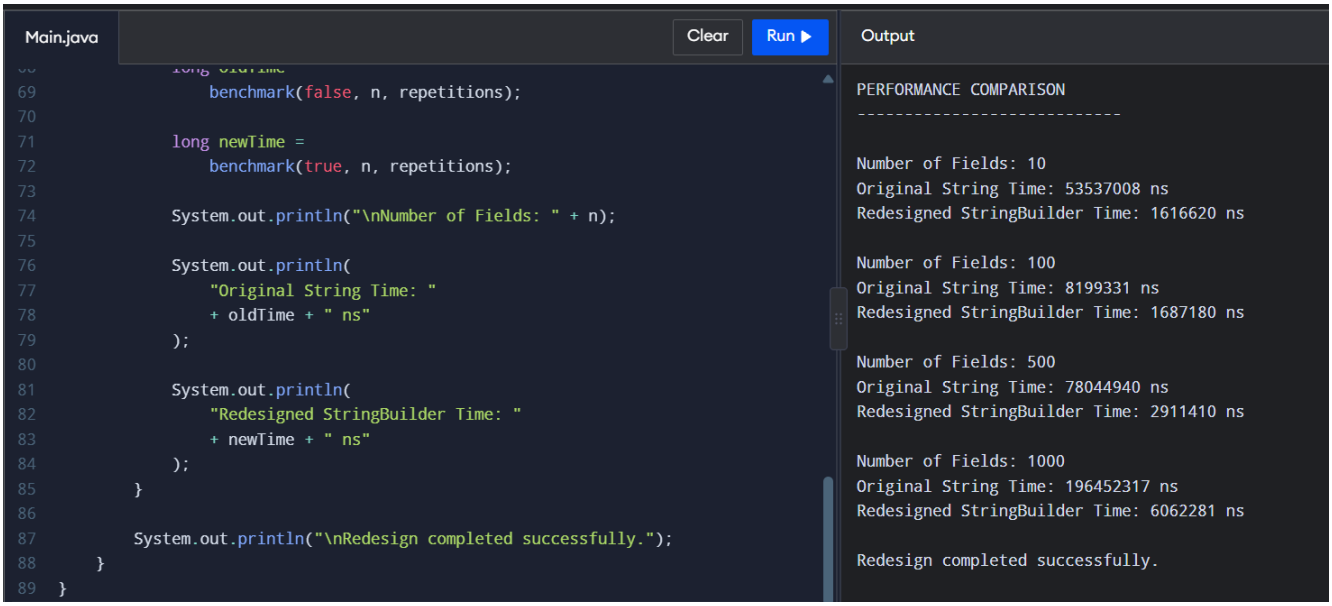
F. Defend the final design

The redesign is successful if repeated measurements show lower construction time for larger inputs while output remains identical. If the measured result is inconsistent, repeat the experiment after warm-up and reduce background activity. String concatenation may remain acceptable for a few fixed pieces of text because it is simple and readable. StringBuilder is the preferred choice for repeated appends in loops.

PROGRAM:

```
public class Main {
    static volatile int result;
    // Original approach using String
    static String original(int fields) {
        String text = "";
        for (int i = 0; i < fields; i++) {
            text = text + "Field" + i + ";";
        }
        return text;
    }
    // Redesigned approach using StringBuilder
    static String redesigned(int fields) {
        StringBuilder text = new StringBuilder();
        for (int i = 0; i < fields; i++) {
            text.append("Field");
            text.append(i);
            text.append(";");
        }
        return text.toString();
    }
    // Calculate execution time
    static long benchmark(
```

```
            boolean useBuilder,
            int fields,
            int repetitions) {
        long start = System.nanoTime();
        for (int i = 0; i < repetitions; i++) {
            if (useBuilder) {
                result = redesigned(fields).length();
            } else {
                result = original(fields).length();
            }
        }
        return System.nanoTime() - start;
    }
    public static void main(String[] args) {
        int[] sizes = {10, 100, 500, 1000};
        int repetitions = 100;
        System.out.println("PERFORMANCE COMPARISON");
        System.out.println("----------------------------");
        for (int n : sizes) {
            long oldTime =
                benchmark(false, n, repetitions);
            long newTime =
                benchmark(true, n, repetitions);
            System.out.println("\nNumber of Fields: " + n);
            System.out.println(
                "Original String Time: "
                + oldTime + " ns"
            );
            System.out.println(
                "Redesigned StringBuilder Time: "
                + newTime + " ns"
            );
        }
        System.out.println("\nRedesign completed successfully.");
    }
}
```



CONCLUSION

This assignment demonstrates that selecting a Java class should depend on the operations and workload of the application. StringBuilder is appropriate for repeated text construction; ArrayList is effective for indexed playlist access; ArrayDeque suits double-ended queues; HashSet requires a correct equals/hashCode contract; TreeMap and LinkedHashMap provide predictable report order; Comparator and generics enable reusable ranking; and PriorityQueue supports severity-based triage with a stable arrival tie-breaker.

The final design decision must be supported by measured results from the student's own execution environment. The benchmark tables in this report are intentionally left for actual measurements. The Java source files provide a starting implementation for collecting that evidence.